

Race Condition ի՞նչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է race condition-ը՝ ինչ է, ինչու է լինում, իրական օրինակներ (սխալ + ճիշտ լուծում), ինչպես հայտնաբերել, և հարցազրույցի հարցեր:

1. Ի՞նչ է Race Condition-ը

Race condition-ը լինում է, երբ մի քանի thread միասին մուտք ունեն (ու փոխում են) նույն **shared data-ն**, ու արդյունքը կախված է thread-ների կատարողական կարգից (որ մեկը առաջ հասնի):

Արդյունքը «մրցում» է (race) thread-ների միջև -> անկանխատեսելի

```
int counter = 0;    // shared

void increment() {
    counter++;      // ՈՉ atomic: read -> +1 -> write
}

// 2 thread -> արդյունքը կարող է սխալ լինել
```

2. Ինչու է լինում — ներհատուկ

`counter++` -ը իրոք 3 մեկուսի քայլ է:

1. read counter (օրինակ 5)
2. +1 (6)
3. write counter (6)

Երկու thread միասին.

```
Thread A: read 5
Thread B: read 5    <- դեռ 5!
Thread A: write 6
Thread B: write 6    <- պետք էր 7, բայց 6 (մեկ increment կորավ)
```

Սա «lost update» է -> race condition:

3. Իրական օրինակ (ՄԽԱԼ կոդ)

```
#include <thread>
int counter = 0;

void work() {
    for (int i = 0; i < 100000; i++)
        counter++;           // RACE CONDITION
}

int main() {
    std::thread t1(work), t2(work);
    t1.join(); t2.join();
    cout << counter;         // ՄՊԱՍԵԼԻՔ 200000, բայց <200000 (անկանխատեսելի)
}
```

4. Լուծում 1 — Mutex

```
#include <mutex>
int counter = 0;
std::mutex mtx;

void work() {
    for (int i = 0; i < 1000000; i++) {
        std::lock_guard<std::mutex> lock(mtx);
        counter++;           // միայն մեկ thread միանգամից
    }
}
// հիմա counter == 2000000 (ճիշտ)
```

5. Լուծում 2 — Atomic (ավելի արագ)

```
#include <atomic>
std::atomic<int> counter{0};

void work() {
    for (int i = 0; i < 1000000; i++)
        counter++;           // ATOMIC -- անբաժանելի, չկա race
}
// counter == 2000000, առանց mutex-ի (ավելի արագ պարզ կառուցվածքների համար)
```

Փոքր operation-ների (counter++) համար `atomic`-ը ավելի արագ է քան mutex. (Տես atomic presentation.)

6. Data Race vs Race Condition

Data Race: երկու thread synchronization-ից առանց նույն memory մուտք (առնվազն մեկը write) -> UNDEFINED BEHAVIOR

Race Condition: արդյունքը կախված է thread-ների կարգից (լոգիկական սխալ)

Data race-ը միշտ սխալ է (UB); race condition-ը ավելի լայն հասկացություն (լոգիկական bug):

7. Ինչպես հայտնաբերել

```
ThreadSanitizer (TSan): g++ -fsanitize=thread ...  
    -> հայտնաբերում data race-ները runtime-ում  
Helgrind (Valgrind):    valgrind --tool=helgrind  
Code review:            shared mutable data + synchronization-ի բացակայություն
```

8. Խուսափելու ձևերը

1. Mutex / lock_guard -> critical section
2. Atomic -> փոքր operation (counter, flag)
3. Immutable data -> չկա փոխում -> չկա race
4. Thread-local data -> ամեն thread իր copy-ն
5. Չխառնել shared mutable state (ավելի լավ design)

9. Հարցազրույցի հարցեր և պատասխաններ

Race condition ի՞նչ է:

Մի քանի thread միասին փոխում են նույն shared data; արդյունքը կախված thread-ների կարգից -> անկանխատեսելի:

Ինչո՞ւ counter++ race է:

Ոչ atomic -- read/+1/write 3 քայլ; thread-ներ խառնվում -> lost update:

Ինչպես լուծել:

Mutex (lock_guard) կամ atomic; immutable/thread-local data:

Data race vs race condition:

Data race 'sync-ից առանց նույն memory (write) -> UB. *Race condition* 'լոգիկական, արդյունք կարգից կախված:

Ինչպես հայտնաբերել:

ThreadSanitizer (-fsanitize=thread), Helgrind, code review:

Mutex vs atomic race-ի համար:

Փոքր operation (counter/flag) -> atomic (արագ); բարդ critical section -> mutex:

Ամփոփում (Cheat Sheet)

RACE CONDITION = մի քանի thread նույն shared data փոխում -> արդյունք կարգից կախված

ԻՆՉՈՒ: counter++ ոչ atomic (read/+1/write) -> lost update

ԼՈՒԾՈՒՄ:

mutex / lock_guard -> critical section

atomic -> փոքր operation (արագ)

immutable / thread-local data

DATA RACE (UB) vs RACE CONDITION (լոգիկական bug)

ՀԱՅՏՆԱԳԵՐԵԼ: ThreadSanitizer (-fsanitize=thread) | Helgrind